

04_Spark_Practica_Ventas

March 12, 2026

1 Práctica 1: E-Commerce y Agregaciones Masivas en Capas

Dataset: Ecommerce Behavior Data (Millones de interacciones) **Tecnología:** Apache Spark (Modo Local) sobre HDFS

En esta práctica vamos a procesar varios millones de registros reales aplicando la **Arquitectura Medallón**. El dataset se compone de **múltiples archivos CSV** a la vez.

¡ATENCIÓN! - Posible Error 403 Forbidden de Kaggle > Si al ejecutar la celda de Kaggle te da un `HTTPError: 403 Client Error: Forbidden`, significa que tu archivo `kaggle.json` no tiene credenciales válidas, o que has superado el límite de descargas de la API. ¡Asegúrate de haber generado un nuevo token en Kaggle.com!

```
[1]: from pyspark.sql import SparkSession
     from pyspark.sql.functions import col, sum, count, desc, when, trim
```

```
[2]: spark = (SparkSession.builder
            .appName("Practica_Ventas_Kaggle")
            .master("local[*]")
            .config("spark.driver.memory", "4g")
            .config("spark.sql.shuffle.partitions", "16")
            .getOrCreate()
        )

# Ajusta particiones para local (muy importante)
# En local, el default puede ser malo (demasiadas o muy pocas tareas).
# Si tienes 8 núcleos (ejemplo)
# spark = (
#     SparkSession.builder
#     .appName("Intro_Spark")
#     .master("local[*]")
#     .config("spark.sql.shuffle.partitions", "16") # prueba 8, 16, 32
#     .getOrCreate()
# )
# Regla práctica
# dataset grande + local → prueba 8 / 16 / 32
# - si pones demasiado alto → overhead
# - si pones demasiado bajo → poco paralelismo
```

```
# Ejemplo: muestra la versión de PySpark
try:
    print(f"Versión de PySpark {spark.version}.")
except NameError:
    print("SparkSession no inicializada; ejecuta la celda de creación de spark.
    ↪")
```

Versión de PySpark 3.5.0.

1.0.1 Bronce: Ingesta Masiva

Este dataset es gigante (varios GB) y contiene archivos CSV que representan diferentes meses (Octubre, Noviembre...). HDFS brillará aquí guardándolos distribuidos, y Spark los leerá todos a la vez.

```
[3]: !pip install -q opendatasets
import opendatasets as od

# Dataset masivo de comportamiento en Ecommerce
dataset_url = "https://www.kaggle.com/datasets/mkechinov/
    ↪ecommerce-behavior-data-from-multi-category-store"

od.download(dataset_url)

# Ingestamos TODOS los CSV (*.csv) a una única ruta en Bronce.
!hdfs dfs -mkdir -p /data/bronze/ecommerce/
!hdfs dfs -put -f ecommerce-behavior-data-from-multi-category-store/*.csv /data/
    ↪bronze/ecommerce/
```

Please provide your Kaggle credentials to download this dataset. Learn more:

<http://bit.ly/kaggle-creds>

Your Kaggle username:

antonioferrermartnez

Your Kaggle Key:

.....

Dataset URL: <https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store>

Downloading ecommerce-behavior-data-from-multi-category-store.zip to
./ecommerce-behavior-data-from-multi-category-store

100%| | 4.29G/4.29G [13:24<00:00, 5.73MB/s]

1.0.2 Plata: Estandarización

Uniremos todos los CSVs y los validaremos (quitando basuras/nulos) para convertirlos a Parquet.

```
[4]: ruta_bronze = "hdfs://namenode:9000/data/bronze/ecommerce/*.csv"

# 1. Leer un directorio entero de CSV con millones de filas haciendo que
    ↳ adivine el esquema
df_ventas = spark.read.option("header", "true").option("inferSchema", "true").
    ↳ csv(ruta_bronze)

total = df_ventas.count()
print(f"Total de registros: {total:,}")
```

Total de registros: 109,950,743

```
[5]: # VER SI ALGUN ID TIENE NULOS
# Ver una sola columna
print("user_id nulos:", df_ventas.filter(col("user_id").isNull()).count())
# Ver varias a la vez:
id_cols = ["product_id", "category_id", "user_id", "user_session"]

df_ventas.select([
    count(when(col(c).isNull(), c)).alias(f"{c}_nulls")
    for c in id_cols
]).show()

# Extra: detectar nulos o vacíos (strings)
df_ventas.filter(
    col("user_session").isNull() | (trim(col("user_session")) == "")
).show(20, truncate=False)
```

user_id nulos: 0

```
+-----+-----+-----+-----+
|product_id_nulls|category_id_nulls|user_id_nulls|user_session_nulls|
+-----+-----+-----+-----+
|              0|              0|              0|              12|
+-----+-----+-----+-----+
```



```
+-----+-----+-----+-----+-----+
|event_time      |event_type|product_id|category_id      |category_code
|brand |price |user_id |user_session|
+-----+-----+-----+-----+-----+
|2019-11-09 15:32:27|cart      |19700004  |2053013559104766575|NULL
|kabrita|37.77  |539704497|NULL              |
|2019-11-09 17:15:24|cart      |1005083   |
|2053013555631882655|electronics.smartphone      |honor  |566.27 |568843390|NULL
|
|2019-11-13 04:02:03|cart      |4804008   |
```

```

|2053013554658804075|electronics.audio.headphone|bluedio|97.81 |570411102|NULL
|
|2019-11-13 07:18:35|cart |1004767
|2053013555631882655|electronics.smartphone |samsung|243.51 |570878749|NULL
|
|2019-11-23 12:53:50|cart |7600528 |2053013552821698803|NULL
|tp-link|16.73 |575357602|NULL |
|2019-11-25 05:04:42|cart |1802104
|2053013554415534427|electronics.video.tv |arg |360.09 |573722572|NULL
|
|2019-11-25 07:03:38|cart |21403753
|2053013561579406073|electronics.clocks |NULL |181.47 |576301354|NULL
|
|2019-11-26 07:48:12|cart |4804718
|2053013554658804075|electronics.audio.headphone|apple |334.58 |576935861|NULL
|
|2019-11-27 07:02:20|cart |12719553 |2053013553559896355|NULL
|NULL |55.6 |577167045|NULL |
|2019-11-29 17:25:56|cart |1005116
|2053013555631882655|electronics.smartphone |apple |1001.42|579123407|NULL
|
|2019-10-06 14:26:10|cart |1801723
|2053013554415534427|electronics.video.tv |tcl |135.65 |557388939|NULL
|
|2019-10-25 10:36:14|cart |1004767
|2053013555631882655|electronics.smartphone |samsung|246.52 |549825742|NULL
|
+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+

```

```
[6]: df_ventas.show(5)
```

```

+-----+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+
|          event_time|event_type|product_id|          category_id|
category_code| brand| price| user_id|          user_session|
+-----+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+
|2019-11-01 00:00:00|      view|  1003461|2053013555631882655|electronics.smart
...|xiaomi|489.07|520088904|4d3b30da-a5e4-49d...|
|2019-11-01 00:00:00|      view|  5000088|2053013566100866035|appliances.sewing
...|janome|293.65|530496790|8e5f4f83-366c-4f7...|
|2019-11-01 00:00:01|      view|  17302664|2053013553853497655|
NULL| creed| 28.31|561587266|755422e7-9040-477...|
|2019-11-01 00:00:01|      view|
3601530|2053013563810775923|appliances.kitche...|
lg|712.87|518085591|3bfb58cd-7892-48c...|

```

```
|2019-11-01 00:00:01|      view|    1004775|2053013555631882655|electronics.smart
...|xiaomi|183.27|558856683|313628f1-68b8-460...|
+-----+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
only showing top 5 rows
```

```
[7]: # 2. Transformación principal: Filtra las interacciones de compras defectuosas
      ↪(precios nulos) y eventos que no sean compras
df_plata = df_ventas.na.drop(subset=["price", "brand"]).
      ↪filter(col("event_type") == "purchase")
df_plata.show(5)
```

```
+-----+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
|      event_time|event_type|product_id|      category_id|
category_code|    brand| price|  user_id|      user_session|
+-----+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
|2019-11-01 00:01:04|  purchase|
1005161|2053013555631882655|electronics.smart...|
xiaomi|211.92|513351129|e6b7ce9b-1938-4e2...|
|2019-11-01 00:04:51|  purchase|
1004856|2053013555631882655|electronics.smart...|
samsung|128.42|562958505|0f039697-fedc-40f...|
|2019-11-01 00:05:34|  purchase| 26401669|2053013563651392361|
NULL|  lucente|109.66|541854711|c41c44d5-ef9b-41b...|
|2019-11-01 00:06:33|  purchase|
1801881|2053013554415534427|electronics.video.tv|  samsung|
488.8|557746614|4d76d6d3-fff5-488...|
|2019-11-01 00:06:34|  purchase|  5800823|2053013553945772349|electronics.audio
...|nakamichi|123.56|514166940|8ef5214a-86ad-4d0...|
+-----+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
only showing top 5 rows
```

```
[8]: # 3. Escribir a Plata
ruta_silver = "hdfs://namenode:9000/data/silver/ecommerce/"
df_plata.repartition(16).write.mode("overwrite").parquet(ruta_silver)
print("Escritura a plata realizada")
```

Escritura a plata realizada

1.0.3 Oro: Transformaciones de Negocio (KPIs)

```
[9]: # 1. Leemos de nuestra Plataforma Silver
df_silver = spark.read.parquet(ruta_silver)

# 2. Regla de Negocio: ¿Qué marca (brand) ha facturado más dinero sumando todos
    ↪ los meses?
df_top_ventas = df_silver.groupBy("brand").agg(sum("price").
    ↪ alias("facturacion_total")).orderBy(desc("facturacion_total"))

# 3. Servir a BI
ruta_gold = "hdfs://namenode:9000/data/gold/top_ventas_ecommerce/"
df_top_ventas.write.mode("overwrite").parquet(ruta_gold)
df_top_ventas.show(10)
```

```
+-----+-----+
| brand| facturacion_total|
+-----+-----+
| apple|2.3872179370001447E8|
|samsung|1.0127741348000082E8|
| xiaomi| 2.04538992500001E7|
| huawei| 9664104.08999993|
| lg| 8626906.719999976|
| acer| 6924026.049999971|
|lucente| 6651658.939999944|
| sony| 6341082.980000027|
| oppo| 5901500.520000067|
| lenovo| 4450744.829999997|
+-----+-----+
```

only showing top 10 rows

1.0.4 1) Usa un formato columnar (Parquet) en vez de CSV

Si estás leyendo CSV, ese suele ser el mayor cuello de botella.

- **CSV** = lento (parseo, tipos, texto)
- **Parquet** = mucho más rápido (columnar, comprimido, schema)

```
[10]: # Ejemplo filtro de nulos sobre df_silver
id_cols = ["product_id", "category_id", "user_id", "user_session"]

df_silver.select([
    count(when(col(c).isNull(), c)).alias(f"{c}_nulls")
    for c in id_cols
]).show()
```

```
+-----+-----+-----+-----+
```

product_id_nulls	category_id_nulls	user_id_nulls	user_session_nulls
0	0	0	0

1.0.5 2) Define esquema manualmente (no inferSchema=True)

inferSchema en archivos grandes hace una pasada extra y puede ser lento.

Mejor:

```
[11]: from pyspark.sql.types import *

schema = StructType([
    StructField("event_time", StringType(), True),
    StructField("event_type", StringType(), True),
    StructField("product_id", LongType(), True),
    StructField("category_id", LongType(), True),
    StructField("category_code", StringType(), True),
    StructField("brand", StringType(), True),
    StructField("price", DoubleType(), True),
    StructField("user_id", LongType(), True),
    StructField("user_session", StringType(), True),
])

ruta_bronze = "hdfs://namenode:9000/data/bronze/ecommerce/*.csv"

# 1. Leer un directorio entero de CSV con millones de filas haciendo que
#    adivine el esquema
# df_ventas = spark.read.option("header", "true").option("inferSchema", "true").
#    csv(ruta_bronze)

df = spark.read.csv(ruta_bronze, header=True, schema=schema)
df.show(10)
```

event_time	event_type	product_id	category_id	category_code	brand	price	user_id	user_session
2019-11-01 00:00:...	view	1003461	2053013555631882655	electronics.smart...	xiaomi	489.07	520088904	4d3b30da-a5e4-49d...
2019-11-01 00:00:...	view	5000088	2053013566100866035	appliances.sewing...	janome	293.65	530496790	8e5f4f83-366c-4f7...
2019-11-01 00:00:...	view	17302664	2053013553853497655					

```

NULL| creed| 28.31|561587266|755422e7-9040-477...|
|2019-11-01 00:00:...| view|
3601530|2053013563810775923|appliances.kitche...|
lg|712.87|518085591|3bfb58cd-7892-48c...|
|2019-11-01 00:00:...| view|
1004775|2053013555631882655|electronics.smart...|
xiaomi|183.27|558856683|313628f1-68b8-460...|
|2019-11-01 00:00:...| view| 1306894|2053013558920217191|
computers.notebook| hp|360.09|520772685|816a59f3-f5ae-4cc...|
|2019-11-01 00:00:...| view| 1306421|2053013558920217191|
computers.notebook| hp|514.56|514028527|df8184cc-3694-454...|
|2019-11-01 00:00:...| view| 15900065|2053013558190408249|
NULL| rondell| 30.86|518574284|5e6ef132-4d7c-473...|
|2019-11-01 00:00:...| view| 12708937|2053013553559896355|
NULL|michelin| 72.72|532364121|0a899268-31eb-46d...|
|2019-11-01 00:00:...| view|
1004258|2053013555631882655|electronics.smart...|
apple|732.07|532647354|d2d3d2c6-631d-489...|
+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
only showing top 10 rows

```

```

[12]: # Muestra cuántas particiones tiene actualmente el DataFrame.
      # Esto te ayuda a entender cómo Spark dividirá el trabajo en tareas paralelas.
      print(df.rdd.getNumPartitions())

      # Reorganiza el DataFrame para que tenga exactamente 16 particiones.
      # Spark redistribuye los datos (shuffle), lo que puede mejorar el paralelismo
      # si antes había muy pocas particiones o estaban mal distribuidas.
      df = df.repartition(16)

```

110

1.1 Explicación

1.1.1 print(df.rdd.getNumPartitions())

- Devuelve el **número actual de particiones** del DataFrame.
- En Spark, las particiones son “trozos” de datos que se procesan en paralelo.
- Si tienes muy pocas, no aprovechas los cores.
- Si tienes demasiadas, hay overhead (muchas tareas pequeñas).

1.1.2 df = df.repartition(16)

- Fuerza al DataFrame a tener **16 particiones**.

- Esto puede mejorar rendimiento en operaciones posteriores (`groupBy`, `join`, etc.) si 16 encaja mejor con tu máquina.
- **Ojo:** `repartition()` suele implicar **shuffle** (mueve datos entre particiones), así que no conviene usarlo sin necesidad.

1.2 Cuándo usarlo

- Antes de operaciones pesadas si ves mala distribución / pocas particiones.
- Si quieres ajustar el paralelismo en local (por ejemplo 8, 16, 32).
- No lo uses en cada paso del pipeline, porque el shuffle cuesta tiempo.

1.3 Avanzado

```
[13]: from pyspark.sql import SparkSession
from pyspark import StorageLevel

# =====
# 1) Crear sesión de Spark (local)
# =====
spark = (
    SparkSession.builder
        .appName("Análisis_Local_Spark")
        .master("local[*]") # Usa todos los núcleos disponibles de tu PC
        .config("spark.driver.memory", "8g") # Memoria para Spark (ajusta según tu
        ↪ RAM)
        .config("spark.sql.shuffle.partitions", "16")
        # Número de particiones por defecto en operaciones pesadas (groupBy, join,
        ↪ orderBy)
        .config("spark.sql.adaptive.enabled", "true")
        # Spark puede ajustar particiones automáticamente en algunos casos (AQE)
        .getOrCreate()
)

# =====
# 2) Leer datos (ejemplo)
# =====
df = spark.read.parquet(ruta_silver)
# Mejor usar Parquet (más rápido que CSV)

# =====
# 3) Ver cuántas particiones tiene el DataFrame
# =====
print("Particiones actuales:", df.rdd.getNumPartitions())
# Esto te dice en cuántos "trozos" Spark dividirá el trabajo en paralelo
```

```

# =====
# 4) Ajustar particiones si hace falta
# =====
df = df.repartition(16)
# Reparte los datos en 16 particiones
# Útil si quieres mejorar paralelismo para operaciones pesadas
# Ojo: repartition hace shuffle (mueve datos), así que no lo uses muchas veces
# ↪ sin necesidad

print("Particiones después de repartition:", df.rdd.getNumPartitions())

# =====
# 5) Cachear solo si lo vas a reutilizar varias veces
# =====
df.persist(StorageLevel.MEMORY_AND_DISK)
# Guarda el DataFrame en memoria (y en disco si no cabe)
# Bueno si vas a hacer varios análisis sobre el mismo df

df.count()
# Acción para "materializar" el cache (sin esto, todavía no se guarda realmente)

# =====
# 6) Ejemplo de análisis
# =====
df.groupBy("event_type").count().show()
# groupBy suele hacer shuffle; por eso ayuda tener bien configuradas las
# ↪ particiones

```

Particiones actuales: 8

Particiones después de repartition: 16

```

+-----+-----+
|event_type|  count|
+-----+-----+
|  purchase|1528301|
+-----+-----+

```

1.4 Explicación fácil (qué hace cada cosa)

1.4.1 local[*]

Usa todos los núcleos de tu computador.

Más paralelo = mejor uso de tu CPU.

1.4.2 spark.driver.memory = 8g

Le da memoria a Spark.

Si tienes poca memoria, Spark va lento o puede fallar.

1.4.3 `spark.sql.shuffle.partitions = 16`

Controla cuántas particiones usa Spark en operaciones pesadas (`groupBy`, `join`, etc.).
En local, 16 suele ir mejor que 200 (que muchas veces es demasiado).

1.4.4 `df.rdd.getNumPartitions()`

Te dice cuántas particiones tiene el `DataFrame`.
Sirve para entender si estás usando pocas o demasiadas.

1.4.5 `df.repartition(16)`

Reorganiza los datos en 16 particiones.
Ayuda a repartir mejor el trabajo, pero cuesta (`shuffle`).

1.4.6 `persist(MEMORY_AND_DISK)`

Guarda el `DataFrame` para reutilizarlo varias veces.
Evita recalcular todo desde cero en cada operación.

1.5 Regla fácil para recordar

- **Explorar rápido** → usa muestra (`sample`)
- **Operaciones pesadas** → revisa particiones
- **Repetir análisis** → usa `cache/persist`
- **Velocidad de lectura** → usa **Parquet**

```
[14]: # Cerrar la sesión de Spark al finalizar
try:
    spark.stop()
    print("SparkSession detenida.")
except Exception as e:
    print("No se pudo detener SparkSession:", e)
```

SparkSession detenida.

```
[ ]:
```